COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/15/2026	**DATE SUBMITTED:**	10/16/2026
NAME:	Almirol Joel A Clifford Lee C. Doplayna	**INSTRUCTOR:**	Engr. Jimlord M Quejado

ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

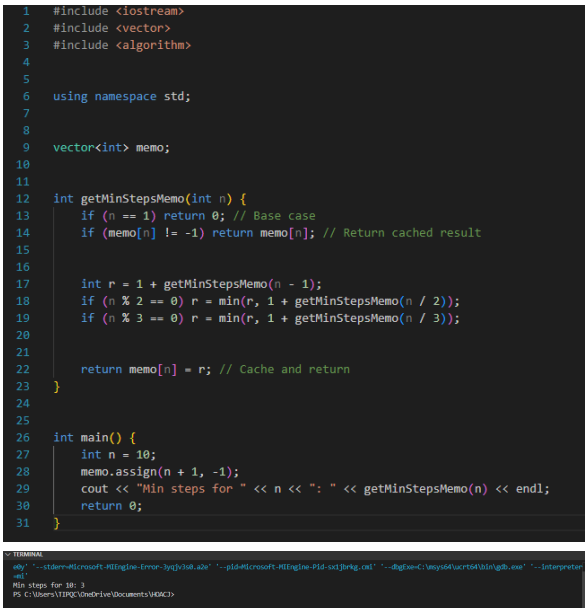
ILOA

Strategy	Algorithm	Analysis
Recursion	**Factorial Computation (find_fact) / Recursive Binary Search**	**This function calls itself with a reduced argument value (n-1), getting closer to a specific base case (n=1).**
Brute Force	**Linear Search / Password Cracker**	**Examines each element/candidate within the entire search space, in sequence, without any pruning until a solution is found.**
Backtracking	**N-Queens Problem / Maze Solver**	**Constructs the solution gradually step-by-step through the state space tree. In case any path breaks constraint, the algorithm backtracks to explore other possibilities.**
Greedy	**Dijkstra's Algorithm / Fractional Knapsack**	**The locally optimum solution is selected at each step by choosing the smallest distance or value density without looking back on previous choices.**
Divide-and-Conquer	**Merge Sort / Quick Sort**	**Divides the array recursively into smaller arrays, solves each independently, and merges them to generate the solution.**

Output

Strategy	Algorithm	Analysis
Recursion	**Factorial Computation (find_fact) / Recursive Binary Search**	**This function calls itself with a reduced argument value (n-1), getting closer to a specific base case (n=1).**
Brute Force	**Linear Search / Password Cracker**	**Examines each element/candidate within the entire search space, in sequence, without any pruning until a solution is found.**
Backtracking	**N-Queens Problem / Maze Solver**	**Constructs the solution gradually step-by-step through the state space tree. In case any path breaks constraint, the algorithm backtracks to explore other possibilities.**
Greedy	**Dijkstra's Algorithm / Fractional Knapsack**	**The locally optimum solution is selected at each step by choosing the smallest distance or value density without looking back on previous choices.**
Divide-and-Conquer	**Merge Sort / Quick Sort**	**Divides the array recursively into smaller arrays, solves each independently, and merges them to generate the solution.**

Table 12-1. Algorithmic Strategies and Examples

Screenshot	
Analysis	

```cpp
#include <iostream>
#include <vector>
#include <algorithm>


using namespace std;


vector<int> memo;


int getMinStepsMemo(int n) {
    if (n == 1) return 0; // Base case
    if (memo[n] != -1) return memo[n]; // Return cached result


    int r = 1 + getMinStepsMemo(n - 1);
    if (n % 2 == 0) r = min(r, 1 + getMinStepsMemo(n / 2));
    if (n % 3 == 0) r = min(r, 1 + getMinStepsMemo(n / 3));


    return memo[n] = r; // Cache and return
}


int main() {
    int n = 10;
    memo.assign(n + 1, -1);
    cout << "Min steps for " << n << ": " << getMinStepsMemo(n) << endl;
    return 0;
}
```

```
Min steps for 10: 3
PS C:\Users\TIPQC\OneDrive\Documents\HOAC>
```

Table 12-2. Memoization Implementation

Screenshot	
Analysis	

```cpp
#include <iostream>
#include <vector>
#include <algorithm>


using namespace std;


int getMinStepsDP(int n) {
    vector<int> dp(n + 1, 0);
    dp[1] = 0; // Base case


    for (int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0) dp[i] = min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0) dp[i] = min(dp[i], 1 + dp[i / 3]);
    }
    return dp[n];
}


int main() {
    int n = 10;
    cout << "Min steps for " << n << ": " << getMinStepsDP(n) << endl;
    return 0;
}
```

```
PS C:\Users\TIPQC\OneDrive\Documents\HOAC> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\TIPQC\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-m2g4sdpl.fbx' '--stdout=Microsoft-MIEngine-Out-0kotw03b.s3k' '--stderr=Microsoft-MIEngine-Error-nzpna43f.4ct' '--pid=Microsoft-MIEngine-Pid-fvvgd14s.jtp' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Min steps for 10: 3
```

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

PSEUDOCODE:

```
FUNCTION countPaths(row, column, remainingCost)

    Subtract current cell cost from remainingCost

    IF remainingCost < 0
        RETURN 0

    IF current cell is the destination
        RETURN 1 if remainingCost is 0, otherwise 0

    IF result is already stored in memo
        RETURN stored result

    Count paths by moving right
    Count paths by moving down

    Store the sum of both results in memo

    RETURN stored result
END FUNCTION
```

CODE:

supplementary.cpp > ...

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

long long countPaths(int i, int j, int remaining,
                     vector<vector<int>>& matrix,
                     vector<vector<vector<long long>>>& memo,
                     int M, int N) {

    remaining -= matrix[i][j];

    if (remaining < 0)
        return 0;

    if (i == M - 1 && j == N - 1)
        return (remaining == 0) ? 1 : 0;

    if (memo[i][j][remaining] != -1)
        return memo[i][j][remaining];

    long long right = 0;
    long long down = 0;

    if (j + 1 < N)
        right = countPaths(i, j + 1, remaining + matrix[i][j],
                           matrix, memo, M, N);

    if (i + 1 < M)
        down = countPaths(i + 1, j, remaining + matrix[i][j],
                          matrix, memo, M, N);

    return memo[i][j][remaining] = right + down;
}

int main() {
    int M, N, targetCost;

    cout << "Enter number of rows and columns: ";
    cin >> M >> N;

    vector<vector<int>> matrix(M, vector<int>(N));

    cout << "Enter the matrix elements:\n";
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            cin >> matrix[i][j];
        }
```

```
    }

    cout << "Enter the target cost: ";
    cin >> targetCost;

    if (targetCost < 0) {
        cout << "Number of paths: 0\n";
        return 0;
    }

    vector<vector<vector<long long>>> memo(
        M, vector<vector<long long>>(
            N, vector<long long>(targetCost + 1, -1)
        )
    );

    long long result = countPaths(
        0, 0, targetCost + matrix[0][0],
        matrix, memo, M, N
    );

    cout << "Number of paths: " << result << endl;

    return 0;
}
```

OUTPUT:

```
TERMINAL
PS C:\Users\TIPQC\OneDrive\Documents\HOACJ>  ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\TIPQC\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAd
apters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-5kn3rbgv.cgo' '--stdout=Microsoft-MIEngine-Out-sdaxp4hu.4xg' '--stderr=Microsoft-MIE
ngine-Error-nf4nhq31.0kt' '--pid=Microsoft-MIEngine-Pid-sqtb1k22.sbd' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Enter number of rows and columns: 3
3
Enter the matrix elements:
1
2
3
4
5
6
1
2
3
Enter the target cost: 3
Number of paths: 0
PS C:\Users\TIPQC\OneDrive\Documents\HOACJ>
```

Analysis:

Doplayna	**This activity uses Top-Down Dynamic Programming with Memoization to solve the matrix path-finding problem efficiently. The countPaths function uses recursion to explore possible paths by moving right or down while subtracting the cost of each cell from the remaining target cost. It also stores previously computed results in a memoization array to avoid repeated calculations. In this activity, the target path sum was 3, but no valid path was found because all possible paths had a total cost greater than the target value.**
Almirol	**The laboratory algorithm employs Top-Down Dynamic Programming with Memoization for solving the matrix path-finding problem using recursion along with cached state lookups for determining paths efficiently without any unnecessary computations. For each cell in the grid, the recursive function countPaths considers the move to the right or the downward move while deducting the cost of the current cell from the budget, returning 0 if the budget falls below 0 during any intermediate step. Once the specific state in the form of coordinates and the remaining target cost (i, j, \text{remaining}) is calculated, its result is cached in the memo 3D array for efficient retrieval during future lookups in \mathcal{O}(1) time complexity. In this particular execution, the target path sum for the given 3 \times 3 matrix was 3, however, since the cost of the starting cell itself was 1 while the cost of all possible paths across the grid was more than 3, none of the paths met the criterion of satisfying the target cost.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion:

DOPLAYNA:

- After finishing this activity, I learned about different algorithmic strategies and how they are used to solve problems efficiently. I also learned how recursion and memoization help reduce repeated calculations in C++. I still need to improve my coding skills, understand each algorithm better, and practice implementing these strategies to solve problems more easily.

ALMIROL:

-

Rubrics:

T.I.P. S.O. 7 Rubric for Lifelong Learning (1)

Criteria	Ratings						Pts
Acquire and apply new knowledge from outside sources	**6 pts** **Excellent** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently and applies knowledge learned into practice	**5 pts** **Good** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently	**4 pts** **Satisfactory** Look beyond classroom requirements, showing interest in pursuing knowledge independently	**3 pts** **Unsatisfactory** Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	**2 pts** **Poor** Relies on classroom instruction only	**1 pts** **Very Poor** No initiative or interest in acquiring new knowledge	6 pts
Learn independently	**6 pts** **Excellent** Completes an assigned task independently and practices continuous improvement	**5 pts** **Good** Completes an assigned task without supervision or guidance	**4 pts** **Satisfactory** Requires minimal guidance to complete an assigned task	**3 pts** **Unsatisfactory** Requires detailed or step-by-step instructions to complete a task	**2 pts** **Poor** Shows little interest to complete a task independently	**1 pts** **Very Poor** No interest to complete a task independently	6 pts
Critical thinking in the broadest context of technological change	**6 pts** **Excellent** Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	**5 pts** **Good** Evaluate information from a variety of sources; formulates a clear and precise perspective	**4 pts** **Satisfactory** Analyze information from a variety of sources; formulates a clear and precise perspective.	**3 pts** **Unsatisfactory** Apply the gathered information to formulate the problem	**2 pts** **Poor** Gather and summarized the information from a variety of sources but failed to formulate the problem	**1 pts** **Very Poor** Gather information from a variety of sources	6 pts
Creativity and adaptability to new and emerging technologies	**6 pts** **Excellent** Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	**5 pts** **Good** Ideas are creative and adapt the new knowledge to solve a problem or address an issue	**4 pts** **Satisfactory** Ideas are creative in solving a problem or address an issue	**3 pts** **Unsatisfactory** Shows some creative ways to solve the problem	**2 pts** **Poor** Shows initiative and attempt to develop creative ideas to solve the problem	**1 pts** **Very Poor** Ideas are copied or restated from the sources consulted	6 pts
							Total Points: 24